Fundamentals of Programming Languages
Assignment 1
Natural numbers and strings

Mestrado em Engenharia Informática
Faculdade de Ciências da Universidade de Lisboa

2026/2027

You must deliver two solutions: one in SASyLF for sections 1 and 2, the other in Haskell for section 3. A paper-and-pencil solution to sections 1 and 2 is optional.

1 Natural numbers, strings and typing

We define a language of expressions for natural numbers, strings and operations on these.

`e` ::=		*expressions:*
	`0`	*constant zero*
	`succ e`	*successor*
	`e` + `e`	*addition*
	ε	*empty string*
	`Ae`	*string starting with* `A`
	`Be`	*string starting with* `B`
	`len e`	*string length*

Natural numbers are built from `0` and operator `succ`. Strings are built from ε, the empty string, by prefixing a string with letter `A` or `B`; for simplicity, these are the only two letters. Examples of strings are ε, `A`ε, `B`ε, `BABA`ε and `ABBABABAABABAAABABABAB`ε. The length of a string, `len`, denotes a natural number. For example, `len` (`BABA`ε) denotes the natural number `succ` (`succ` (`succ` (`succ` `0`))). We use parentheses to group expressions.

A. Suggest a grammar for the types of expressions. Use metavariable `T` to denote arbitrary types.

B. Present a type system assigning types to expressions. Use judgements of the form $\texttt{e} : \texttt{T}$.

C. Exhibit an expression that is typable according to the type system just defined and another that is untypable. Each of the two expressions must contain both $+$ and `len`. Show that the expressions you have selected are indeed typable and untypable.

D. Show the Uniqueness of Types Lemma: For every expression `e` there is at most one type `T` such that $\texttt{e} : \texttt{T}$.

2 Evaluation and type safety

The dynamics of the expression language is given by a one-step evaluation relation on expressions.

A. Define the predicate is-value by means of rules. Use a judgement of the form `value e`.

B. Define a *one-step evaluation* relation for the expression language. Use a judgement of the form $\texttt{e} \to \texttt{e}$. Evaluate the operands first, left to right, and only then apply the operator. Operators apply to values only: addition to *numerical values* (the values built from `0` and `succ`) and length to *string values* (the values built from ε, `A` and `B`).

- Addition proceeds by analysing its first operand. If `e` is a numerical value, then $\texttt{0} + \texttt{e}$ evaluates in one step to `e`. If $\texttt{succ}\ \texttt{e}_1$ and $\texttt{e}_2$ are numerical values, then $(\texttt{succ}\ \texttt{e}_1) + \texttt{e}_2$ evaluates in one step to $\texttt{succ}\ (\texttt{e}_1 + \texttt{e}_2)$.
- Length proceeds by analysing its operand. Expression $\texttt{len}\ \varepsilon$ evaluates in one step to `0`. If `e` is a string value, then both $\texttt{len}\ (\texttt{Ae})$ and $\texttt{len}\ (\texttt{Be})$ evaluate in one step to $\texttt{succ}\ (\texttt{len}\ \texttt{e})$.
- An operand that is not a value is evaluated in place. For example, if `e` is *not* a value and $\texttt{e} \to \texttt{e}'$, for some $\texttt{e}'$, then $\texttt{0} + \texttt{e}$ evaluates in one step to $\texttt{0} + \texttt{e}'$.

C. Use your rules to derive the following evaluation steps:

$$\begin{aligned}
(\texttt{succ 0}) + (\texttt{succ 0}) &\to \texttt{succ}\ (\texttt{0} + (\texttt{succ 0})) \\
\texttt{succ}\ (\texttt{0} + (\texttt{succ 0})) &\to \texttt{succ}\ (\texttt{succ 0}) \\
\texttt{len}\ (\texttt{AB}\varepsilon) &\to \texttt{succ}\ (\texttt{len}\ (\texttt{B}\varepsilon)) \\
\texttt{succ}\ (\texttt{len}\ (\texttt{B}\varepsilon)) &\to \texttt{succ}\ (\texttt{succ}\ (\texttt{len}\ \varepsilon)) \\
(\texttt{len}\ (\texttt{A}\varepsilon)) + (\texttt{succ}\ (\texttt{succ 0})) &\to (\texttt{succ}\ (\texttt{len}\ \varepsilon)) + (\texttt{succ}\ (\texttt{succ 0}))
\end{aligned}$$

D. An expression `e` is a *normal form* if no evaluation rule applies to it, that is, if there is no $\texttt{e}'$ such that $\texttt{e} \to \texttt{e}'$. Show that every value is a normal form: if `value e`, then there is no $\texttt{e}'$ such that $\texttt{e} \to \texttt{e}'$.

E. Show that the converse does not hold: exhibit an expression that is a normal form but not a value. Show that the expression you have selected is indeed a normal form and that it is indeed not a value.

F. Show the Progress Theorem. If `e : T`, then either `value e` or there exists $\texttt{e}'$ such that $\texttt{e} \to \texttt{e}'$.

G. Conclude from item 2.F that every typable normal form is a value: if `e : T` and `e` is a normal form, then `value e`. Is the expression you have selected in item 2.E typable? Could you have selected a typable one? Justify.

H. Show the Preservation Theorem. If `e : T` and $\texttt{e} \to \texttt{e}'$, then $\texttt{e}' : \texttt{T}$.

3 Implementation

A. Write a `data` declaration to describe expressions. Write Haskell values for the five expressions on the left of the arrows in item 2.C.

B. Write predicate `val` and function `step`, which evaluates an expression in one step. Use these to define a function `eval` that evaluates an expression to a value. Predicate `val` must be total: it must yield `True` or `False` for every expression, typable or not, and never raise an exception. Functions `step` and `eval` may raise an exception (`Exception: Non-exhaustive patterns in function`, for example): `step` when no evaluation rule applies to the given expression, and `eval` when the given expression does not evaluate to a value.

C. Write predicate `normal` that decides whether a given expression is a normal form. The predicate must be total. Then write predicate `valueIsNormal` that checks, for a given expression, the property in item 2.D: if the expression is a value, then it is a normal form. Confirm

that `valueIsNormal` holds for the normal forms of the five expressions in item 3.A; use `eval` to compute these normal forms.

D. Write function `typeof` that computes the type of a given expression if this exists, and raises an exception otherwise (`Exception: Non-exhaustive patterns in function`, for example).

E. Write a Haskell value `stuck` for the expression you have selected in item 2.E. Use `normal` and `val` to confirm that it is indeed a normal form and that it is indeed not a value. What do you obtain from `eval stuck` and from `typeof stuck`? Explain.

Due date: October 18, 2026, 23:59